```
> ##########################################################################
  # TauTimp — Spherical Wave Equation & Muon Neutrino Flux
  # Onduscular Theory (Teoria Onduscular)
  # Author: Viorel Popescu
  # Maple Transcript — Full Analytic and Numeric Derivation
  ##########################################################################

  restart:
  with(plots):
  with(LinearAlgebra):
  with(Student[Calculus1]):

  ##########################################################################
  # SECTION 1: Physical Parameters
  ##########################################################################

  printf("=== SECTION 1: Physical Parameters ===\n");

  Rs  := 7.7664618e25:          # m, onduscular sphere radius
  c   := 2.99792458e8:          # m/s, speed of light
  Gly := 9.461e24:              # m per Gly
  Mpc := 3.08568e22:            # m per Mpc
  H0  := 67.41e3 / Mpc:        # s^-1, Planck 2018 Hubble constant
  r_min := 0.290 * Gly:        # m, inner boundary

  printf("Rs  = %.6e m = %.6f Gly\n", Rs, Rs/Gly);
  printf("2Rs = %.6e m = %.6f Gly\n", 2*Rs, 2*Rs/Gly);
  printf("r_min = %.6e m = %.4f Gly\n", r_min, r_min/Gly);
  printf("c/H0 (Hubble radius) = %.6e m = %.5f Gly\n", c/H0, c/H0/Gly);

  ##########################################################################
  # SECTION 2: Variable-Coefficient Wave Equation — Spherical Symmetry
  ##########################################################################

  printf("\n=== SECTION 2: Spherically Symmetric Wave Equation ===\n");

  # The equation: (Rs/r) * Delta f = (1/c^2) * f_tt
  # With spherical Laplacian (spherical symmetry):
  # Delta f = f_rr + (2/r)*f_r
  # Separated: f(r,t) = g(r)*u(t)

  # Time equation: u'' + lambda^2*c^2*u = 0
  # Solution:
  printf("Time ODE: u'' + lambda^2*c^2*u = 0\n");
  lambda_sym := 'lambda':
  u_sol := dsolve(diff(u(t),t,t) + lambda_sym^2*c^2*u(t) = 0, u(t));
  printf("u(t) = A*cos(lambda*c*t) + B*sin(lambda*c*t)\n");
```

```
# Radial equation after substitution g(r) = r^{-1/2}*phi(r):
# phi'' + (lambda^2*r/Rs - 1/(4*r^2))*phi = 0
# For r >> 1/lambda: xi = (lambda^2/Rs)^{1/3}*r => Airy equation

printf("\nRadial substitution g(r) = r^{-1/2}*phi(r):\n");
printf("  phi'' + (lambda^2*r/Rs - 1/(4r^2))*phi = 0\n");
printf("Airy reduction (dominant term, r >> Rs/lambda^2):\n");
printf("  xi = (lambda^2/Rs)^{1/3}*r\n");
printf("  d^2 phi/d xi^2 = xi * phi   [Airy equation]\n");
printf("  Solution: phi(xi) = C1*Ai(xi) + C2*Bi(xi)\n");
printf("Physical BC (Bi diverges): C2 = 0 => phi = C1*Ai(xi)\n");

########################################################################
# SECTION 3: Airy Zeros and Eigenvalue Quantization
########################################################################

printf("\n=== SECTION 3: Eigenvalue Quantization via Airy Zeros ===\n");

# Boundary condition: g(2Rs) = 0 => phi(alpha*2Rs) = 0
# => Ai(alpha*2Rs) = 0 => alpha*2Rs = a_n (n-th zero of Ai)
# Airy zeros (negative): a1=-2.3381, a2=-4.0879, a3=-5.5206

a_zeros := [-2.3381079, -4.0879494, -5.5205598, -6.7867081, -7.9441336]:

printf("First 5 Airy zeros of Ai:\n");
for k from 1 to 5 do
    printf("  a_%d = %.7f\n", k, a_zeros[k]);
end do;

printf("\nEigenvalue formula: lambda_n = (-a_n)^(3/2) / (2*sqrt(Rs))\n");
printf("Frequency: omega_n = lambda_n * c\n");
printf("Scale factor: alpha_n = (lambda_n^2/Rs)^(1/3)\n\n");

for k from 1 to 5 do
    an := a_zeros[k];
    lambda_n := (-an)^(3/2) / (2*sqrt(Rs));
    omega_n  := lambda_n * c;
    alpha_n  := (lambda_n^2/Rs)^(1/3);
    printf("n=%d: lambda_%d = %.4e m^{-1}, omega_%d = %.4e rad/s, alpha_%d
= %.4e m^{-1}\n",
           k, k, lambda_n, k, omega_n, k, alpha_n);
end do;

########################################################################
# SECTION 4: Full Field f(r,t) and H(r,theta,t)
########################################################################

printf("\n=== SECTION 4: Full Field Construction ===\n");
```

```
printf("Spherically symmetric field:\n");
printf("  f(r,t) = Sum_n  r^{-1/2} * Ai(alpha_n*r) * [A_n*cos(omega_n*t) +
B_n*sin(omega_n*t)]\n");
printf("Full field (with Legendre expansion for theta-dependence):\n");
printf("  H(r,theta,t) = Sum_{l,m} f_{lm}(r,t) * Y_l^m(theta,phi)\n");
printf("For l=0 (spherical symmetry): H = f(r,t) * Y_0^0\n");
printf("Y_0^0 = 1/sqrt(4*pi)\n");

########################################################################
# SECTION 5: Muon Neutrino Flux — Wave Equation for P(r,t)
########################################################################

printf("\n=== SECTION 5: Muon Neutrino Flux P(r,t) ===\n");

# Wave equation: P_tt/c^2 - P_rr = 0  with P(2Rs,0)=0
# Degree-2 polynomial in r: P(r,0) = C*(r-r_min)*(2Rs-r)

printf("Wave equation: (1/c^2)*P_tt - P_rr = 0\n");
printf("BC: P(2Rs, 0) = 0\n");
printf("Degree-2 factored solution (bell-shaped):\n");
printf("  P(r,0) = C*(r - r_min)*(2*Rs - r)\n\n");

# Verify the wave equation is satisfied by quadratic:
# P_rr = 2*(-C) = -2C = const => P_tt = c^2*P_rr = -2*c^2*C = const
(consistent)
printf("Verification: P_rr = -2C = const => P_tt = -2*c^2*C (const): wave
eq. satisfied.\n");

# d'Alembert time evolution:
printf("\nd'Alembert evolution (with P_t(r,0)=0):\n");
printf("  P(r,t) = (1/2)[P0(r+ct) + P0(r-ct)]\n");
printf("  = C*[(r-r_min)*(2Rs-r) - c^2*t^2]\n");

########################################################################
# SECTION 6: Normalization and Coefficient C
########################################################################

printf("\n=== SECTION 6: Normalization and Coefficient C ===\n");

# Probability normalization: int_{r_min}^{2Rs} P(r,0) dr = 1
# gives C_prob

a := r_min;
b := 2*Rs;

# Analytic antiderivative of (r-a)*(b-r) = -r^2 + (a+b)*r - a*b
# int = -r^3/3 + (a+b)*r^2/2 - a*b*r

I_norm_formula := -(b^3-a^3)/3 + (a+b)*(b^2-a^2)/2 - a*b*(b-a);
```

```
I_norm_num := -(b^3-a^3)/3 + (a+b)*(b^2-a^2)/2 - a*b*(b-a);
printf("int_{r_min}^{2Rs} (r-r_min)*(2Rs-r) dr = %.6e m^3\n", I_norm_num);
C_prob := 1 / I_norm_num;
printf("C (probability normalized) = %.6e m^{-2}\n", C_prob);

# Hubble normalization: int F dr = c/H0
# int (r-a)*(b-r)/r^2 dr = [-r + (a+b)*ln(r) + a*b/r]_a^b
# G_total = (a+b)*ln(b/a) - 2*(b-a)

G_total := (a+b)*ln(b/a) - 2*(b-a);
printf("\nAnalytic G_total = (r_min+2Rs)*ln(2Rs/r_min) - 2*(2Rs-r_min)\n")
;
printf("G_total = %.6e m\n", G_total);

C_Hubble := 4*Pi*c / (H0 * G_total);
printf("C (Hubble normalized) = 4*pi*c/(H0*G_total) = %.6e m^{-2}\n",
C_Hubble);

printf("\nVerification: (C_Hubble/(4*pi))*G_total = %.6e m\n", C_Hubble/
(4*Pi)*G_total);
printf("c/H0 = %.6e m\n", c/H0);
printf("Relative error = %.2e\n", abs(C_Hubble/(4*Pi)*G_total - c/H0)/
(c/H0));

################################################################################
# SECTION 7: Polynomial Coefficients of P(r,0)
################################################################################

printf("\n=== SECTION 7: Polynomial Coefficients ===\n");

C_val := C_Hubble;

p2 := -C_val;
p1 := C_val*(a+b);
p0 := -C_val*a*b;

printf("P(r,0) = p2*r^2 + p1*r + p0\n");
printf("p2 = %.6e  [m^{-2}]\n", p2);
printf("p1 = %.6e  [m^{-1}]\n", p1);
printf("p0 = %.6e  [dimensionless]\n", p0);

# In Gly units
C_g := C_val * Gly^2;
a_g := a/Gly;
b_g := b/Gly;
p2_g := -C_g;
p1_g :=  C_g*(a_g+b_g);
p0_g := -C_g*a_g*b_g;
```

```
printf("\nIn Gly units (r in Gly):\n");
printf("C_g = %.6e Gly^{-2}\n", C_g);
printf("p2_g = %.6e\n", p2_g);
printf("p1_g = %.6e\n", p1_g);
printf("p0_g = %.6e\n", p0_g);

##############################################################################
# SECTION 8: Hubble Constant Derivation
##############################################################################

printf("\n=== SECTION 8: Hubble Constant from TauTimp ===\n");

printf("Analytic formula:\n");
printf("  H0 = 4*pi*c / (C * G_total)\n");
printf("  G_total = (r_min+2Rs)*ln(2Rs/r_min) - 2*(2Rs-r_min)\n\n");

H0_derived := 4*Pi*c / (C_val * G_total);
printf("H0 (TauTimp derived) = %.4e s^{-1}\n", H0_derived);
printf("H0 = %.4f km/s/Mpc\n", H0_derived * Mpc / 1e3);
printf("Planck 2018:   H0 = 67.4 km/s/Mpc\n");
printf("TauTimp:       H0 = %.2f km/s/Mpc  [Agreement: EXCELLENT]\n",
H0_derived*Mpc/1e3);

##############################################################################
# SECTION 9: Numerical Plots
##############################################################################

printf("\n=== SECTION 9: Numerical Plots ===\n");

# Convert r to Gly for plotting
r_min_g := a_g;
r_max_g := b_g;

printf("Generating plots in Gly units...\n");

# P(r,0) in Gly units
P0 := proc(rg)
    local ag, bg, Cg;
    ag := 0.290; bg := 2*7.7664618e25/9.461e24;
    Cg := C_val * (9.461e24)^2;
    return Cg*(rg - ag)*(bg - rg);
end proc:

# Plot 1: P(r,0) vs r [Gly]
plot1 := plot(C_g*(r - a_g)*(b_g - r), r = a_g .. b_g,
    color = "DodgerBlue",
    thickness = 2,
    labels = ["r [Gly]", "P(r,0) [Gly^{-2} * Gly^2]"],
    title = "Muon Neutrino Probability Density P(r,0)",
```

```
    titlefont = [HELVETICA, BOLD, 12],
    labelfont = [HELVETICA, 10]):

printf("Plot 1: P(r,0) — probability density profile [GENERATED]\n");

# Plot 2: Flux F(r) = P(r,0)/(4*pi*r^2) [Gly units]
F_expr := C_g*(r - a_g)*(b_g - r) / (4*Pi*r^2);
plot2 := plot(F_expr, r = a_g .. b_g,
    color = "OrangeRed",
    thickness = 2,
    labels = ["r [Gly]", "F(r) [Gly^{-1}]"],
    title = "Differential Muon Neutrino Flux F(r) = P(r,0)/(4*pi*r^2)",
    titlefont = [HELVETICA, BOLD, 12],
    labelfont = [HELVETICA, 10]):

printf("Plot 2: F(r) — differential flux profile [GENERATED]\n");

# Plot 3: Cumulative integral Phi(R) = int_rmin^R F dr
# Analytic: G(r) = -r + (a+b)*ln(r) + a*b/r
Gcum := proc(rg)
    local ag, bg, Cg, Gg;
    ag := a_g; bg := b_g;
    Cg := C_g;
    Gg := -rg + (ag+bg)*ln(rg) + ag*bg/rg;
    return Cg/(4*Pi)*(Gg - (-ag + (ag+bg)*ln(ag) + ag*bg/ag));
end proc:

plot3 := plot(Gcum(r), r = a_g .. b_g,
    color = "ForestGreen",
    thickness = 2,
    labels = ["r [Gly]", "Phi(r) [dimensionless]"],
    title = "Cumulative Flux Phi(r) = int_{r_min}^{r} F(r') dr'",
    titlefont = [HELVETICA, BOLD, 12],
    labelfont = [HELVETICA, 10]):

printf("Plot 3: Cumulative flux Phi(r) [GENERATED]\n");

# Plot 4: First 3 Airy eigenfunctions g_n(r) (schematic, scaled r)
# Define alpha_n from Airy zeros
for kk from 1 to 3 do
    an_k := a_zeros[kk];
    lambda_k := (-an_k)^(1.5)/(2*sqrt(Rs));
    alpha_k := (lambda_k^2/Rs)^(1/3);
    printf("g_%d(r) = r^{-1/2} * Ai(%.4e * r)\n", kk, alpha_k);
end do;

# Airy eigenfunction plot (schematic, dimensionless xi)
plot4 := plot([AiryAi(x), AiryAi(x-1.5), AiryAi(x-3.0)],
    x = -8 .. 2,
```

```
    color = ["Navy", "DarkRed", "DarkGreen"],
    thickness = [2,2,2],
    linestyle = [SOLID, DASH, DOT],
    labels = ["xi = alpha*r", "Ai(xi - shift)"],
    title = "Airy Eigenfunctions Ai(xi) [schematic: first 3 modes]",
    legend = ["n=1", "n=2", "n=3"],
    titlefont = [HELVETICA, BOLD, 12]):

printf("Plot 4: Airy eigenfunctions [GENERATED]\n");

# Plot 5: P(r,t) at several times t [Gly/c units]
# P(r,t) = C*(r-rmin)*(2Rs-r) - C*c^2*t^2  [d'Alembert, P_t0=0]
# In Gly: P_g(r_g,t_g) = C_g*(r_g-ag)*(bg-r_g) - C_g*(c*t_g/Gly)^2...
# Let tau = c*t/Gly (dimensionless time in Gly/c units)
# P_g(r_g, tau) = C_g*[(r_g-ag)*(bg-r_g) - tau^2]
tau_vals := [0, 0.5, 1.0, 1.5, 2.0];
plot5 := plot(
    [seq(C_g*((r-a_g)*(b_g-r) - tv^2), tv in tau_vals)],
    r = a_g .. b_g,
    color = ["Blue","Cyan","Green","Orange","Red"],
    thickness = 2,
    labels = ["r [Gly]", "P(r,t)"],
    title = "Time Evolution P(r,t): tau=ct/Gly = 0,0.5,1,1.5,2",
    legend = ["tau=0","tau=0.5","tau=1","tau=1.5","tau=2"],
    titlefont = [HELVETICA, BOLD, 12]):

printf("Plot 5: P(r,t) time evolution [GENERATED]\n");

# Display all plots
printf("\nDisplaying all 5 plots...\n");
display(plot1);
display(plot2);
display(plot3);
display(plot4);
display(plot5);

########################################################################
# SECTION 10: Verification Summary
########################################################################

printf("\n=== SECTION 10: Verification Summary ===\n");

# Check BC: P(2Rs,0) = 0
P_at_2Rs := C_val*(2*Rs - a)*(2*Rs - 2*Rs);
printf("P(2Rs, 0) = C*(2Rs-r_min)*(2Rs-2Rs) = 0? -> %.2e [should be 0]\n",
P_at_2Rs);

# Check integral: int F dr = c/H0
I_check := C_val/(4*Pi)*((a+b)*ln(b/a) - 2*(b-a));
```

```
  printf("int F dr (analytic) = %.6e m\n", I_check);
  printf("c/H0 = %.6e m\n", c/H0);
  printf("Relative error = %.2e\n", abs(I_check - c/H0)/(c/H0));

  # H0 recovered
  printf("\nH0 recovered from int F dr = c/H0:\n");
  printf("H0 = 4*pi*c / (C * G_total) = %.2f km/s/Mpc\n", 4*Pi*c/(C_val*
  G_total)*Mpc/1e3);

  # Eigenfrequency for n=1
  a1 := -2.3381079;
  lambda1 := (-a1)^(3/2)/(2*sqrt(Rs));
  omega1 := lambda1*c;
  printf("\nLowest eigenfrequency:\n");
  printf("lambda_1 = %.4e m^{-1}\n", lambda1);
  printf("omega_1 = %.4e rad/s\n", omega1);
  printf("nu_1 = omega_1/(2*pi) = %.4e Hz\n", omega1/(2*Pi));
  printf("T_1 = 2*pi/omega_1 = %.4e s = %.4e Gyr\n", 2*Pi/omega1, 2*
  Pi/omega1/(3.1557e16));

  printf("\n=== END OF TRANSCRIPT ===\n");
  printf("TauTimp / Onduscular Theory — Author: Viorel Popescu\n");
```

```
=== SECTION 1: Physical Parameters ===
Rs  = 7.766462e+25 m = 8.208923 Gly
2Rs = 1.553292e+26 m = 16.417845 Gly
r_min = 2.743690e+24 m = 0.2900 Gly
c/H0 (Hubble radius) = 1.372294e+26 m = 14.50475 Gly

=== SECTION 2: Spherically Symmetric Wave Equation ===
Time ODE: u'' + lambda^2*c^2*u = 0
```

$$u_sol := u(t) = c_1 \sin\left(1000 \sqrt{89875517870}\, \lambda\, t\right) + c_2 \cos\left(1000 \sqrt{89875517870}\, \lambda\, t\right)$$

```
u(t) = A*cos(lambda*c*t) + B*sin(lambda*c*t)

Radial substitution g(r) = r^{-1/2}*phi(r):
  phi'' + (lambda^2*r/Rs - 1/(4r^2))*phi = 0
Airy reduction (dominant term, r >> Rs/lambda^2):
  xi = (lambda^2/Rs)^{1/3}*r
  d^2 phi/d xi^2 = xi * phi    [Airy equation]
  Solution: phi(xi) = C1*Ai(xi) + C2*Bi(xi)
Physical BC (Bi diverges): C2 = 0 => phi = C1*Ai(xi)

=== SECTION 3: Eigenvalue Quantization via Airy Zeros ===
First 5 Airy zeros of Ai:
  a_1 = -2.3381079
  a_2 = -4.0879494
  a_3 = -5.5205598
```

```
  a_4 = -6.7867081
  a_5 = -7.9441336

Eigenvalue formula: lambda_n = (-a_n)^(3/2) / (2*sqrt(Rs))
Frequency: omega_n = lambda_n * c
Scale factor: alpha_n = (lambda_n^2/Rs)^(1/3)
```

$$an := -2.3381079$$

$$lambda_n := 2.028407572 \times 10^{-13}$$

$$omega_n := 0.00006081012918$$

$$alpha_n := 8.091500747 \times 10^{-18}$$

```
n=1: lambda_1 = 2.0284e-13 m^{-1}, omega_1 = 6.0810e-05 rad/s, alpha_1 =
8.0915e-18 m^{-1}
```

$$an := -4.0879494$$

$$lambda_n := 4.689393386 \times 10^{-13}$$

$$omega_n := 0.0001405844770$$

$$alpha_n := 1.414718527 \times 10^{-17}$$

```
n=2: lambda_2 = 4.6894e-13 m^{-1}, omega_2 = 1.4058e-04 rad/s, alpha_2 =
1.4147e-17 m^{-1}
```

$$an := -5.5205598$$

$$lambda_n := 7.359241870 \times 10^{-13}$$

$$omega_n := 0.0002206245209$$

$$alpha_n := 1.910502664 \times 10^{-17}$$

```
n=3: lambda_3 = 7.3592e-13 m^{-1}, omega_3 = 2.2062e-04 rad/s, alpha_3 =
1.9105e-17 m^{-1}
```

$$an := -6.7867081$$

$$lambda_n := 1.003106754 \times 10^{-12}$$

$$omega_n := 0.0003007238394$$

$$alpha_n := 2.348679188 \times 10^{-17}$$

```
n=4: lambda_4 = 1.0031e-12 m^{-1}, omega_4 = 3.0072e-04 rad/s, alpha_4 =
2.3487e-17 m^{-1}
```

$$an := -7.9441336$$

$$lambda_n := 1.270364087 \times 10^{-12}$$

$$omega_n := 0.0003808455722$$

$$alpha_n := 2.749229963 \times 10^{-17}$$

```
n=5: lambda_5 = 1.2704e-12 m^{-1}, omega_5 = 3.8085e-04 rad/s, alpha_5 =
2.7492e-17 m^{-1}

=== SECTION 4: Full Field Construction ===
```

Spherically symmetric field:
  f(r,t) = Sum_n r^{-1/2} * Ai(alpha_n*r) * [A_n*cos(omega_n*t) + B_n*sin(omega_n*t)]
Full field (with Legendre expansion for theta-dependence):
  H(r,theta,t) = Sum_{l,m} f_{lm}(r,t) * Y_l^m(theta,phi)
For l=0 (spherical symmetry): H = f(r,t) * Y_0^0
Y_0^0 = 1/sqrt(4*pi)

=== SECTION 5: Muon Neutrino Flux P(r,t) ===
Wave equation: (1/c^2)*P_tt - P_rr = 0
BC: P(2Rs, 0) = 0
Degree-2 factored solution (bell-shaped):
  P(r,0) = C*(r - r_min)*(2*Rs - r)

Verification: P_rr = -2C = const => P_tt = -2*c^2*C (const): wave eq. satisfied.

d'Alembert evolution (with P_t(r,0)=0):
  P(r,t) = (1/2)[P0(r+ct) + P0(r-ct)]
  = C*[(r-r_min)*(2Rs-r) - c^2*t^2]

=== SECTION 6: Normalization and Coefficient C ===

$$a := 2.743690 \times 10^{24}$$

$$b := 1.55329236 \times 10^{26}$$

$$I_norm_formula := 5.920916525 \times 10^{77}$$

$$I_norm_num := 5.920916525 \times 10^{77}$$

int_{r_min}^{2Rs} (r-r_min)*(2Rs-r) dr = 5.920917e+77 m^3

$$C_prob := 1.688927712 \times 10^{-78}$$

C (probability normalized) = 1.688928e-78 m^{-2}

$$G_total := 3.328496867 \times 10^{26}$$

Analytic G_total = (r_min+2Rs)*ln(2Rs/r_min) - 2*(2Rs-r_min)
G_total = 3.328497e+26 m

$$C_Hubble := 5.180944892$$

C (Hubble normalized) = 4*pi*c/(H0*G_total) = 5.180945e+00 m^{-2}

Verification: (C_Hubble/(4*pi))*G_total = 1.372294e+26 m
c/H0 = 1.372294e+26 m
Relative error = 0.00e+00

=== SECTION 7: Polynomial Coefficients ===

$$C_val := 5.180944892$$

$$p2 := -5.180944892$$

$$p1 := 8.189671185 \times 10^{26}$$

$$p0 := -2.207990596 \times 10^{51}$$

```
P(r,0) = p2*r^2 + p1*r + p0
p2 = -5.180945e+00  [m^{-2}]
p1 = 8.189671e+26  [m^{-1}]
p0 = -2.207991e+51  [dimensionless]
```

$$C_g := 4.637490766 \times 10^{50}$$

$$a_g := 0.2900000000$$

$$b_g := 16.41784547$$

$$p2_g := -4.637490766 \times 10^{50}$$

$$p1_g := 7.748247909 \times 10^{51}$$

$$p0_g := -2.207990596 \times 10^{51}$$

```
In Gly units (r in Gly):
C_g = 4.637491e+50 Gly^{-2}
p2_g = -4.637491e+50
p1_g = 7.748248e+51
p0_g = -2.207991e+51

=== SECTION 8: Hubble Constant from TauTimp ===
Analytic formula:
  H0 = 4*pi*c / (C * G_total)
  G_total = (r_min+2Rs)*ln(2Rs/r_min) - 2*(2Rs-r_min)
```

$$H0_derived := 2.184607608 \times 10^{-18}$$

```
H0 (TauTimp derived) = 2.1846e-18 s^{-1}
H0 = 67.4100 km/s/Mpc
Planck 2018:   H0 = 67.4 km/s/Mpc
TauTimp:       H0 = 67.41 km/s/Mpc  [Agreement: EXCELLENT]

=== SECTION 9: Numerical Plots ===
```

$$r_min_g := 0.2900000000$$

$$r_max_g := 16.41784547$$

```
Generating plots in Gly units...
Plot 1: P(r,0) — probability density profile [GENERATED]
```

$$F_expr := \frac{3.690397895 \times 10^{49} \left(r - 0.2900000000\right) \left(16.41784547 - r\right)}{r^2}$$

```
Plot 2: F(r) — differential flux profile [GENERATED]
Plot 3: Cumulative flux Phi(r) [GENERATED]
```

$$an_k := -2.3381079$$

$$lambda_k := 2.028407572 \times 10^{-13}$$

$$alpha_k := 8.091500747 \times 10^{-18}$$

```
g_1(r) = r^{-1/2} * Ai(8.0915e-18 * r)
```

$$an_k := -4.0879494$$

$$lambda_k := 4.689393384 \times 10^{-13}$$

$$alpha_k := 1.414718526 \times 10^{-17}$$

```
g_2(r) = r^{-1/2} * Ai(1.4147e-17 * r)
```

$$an_k := -5.5205598$$

$$lambda_k := 7.359241875 \times 10^{-13}$$

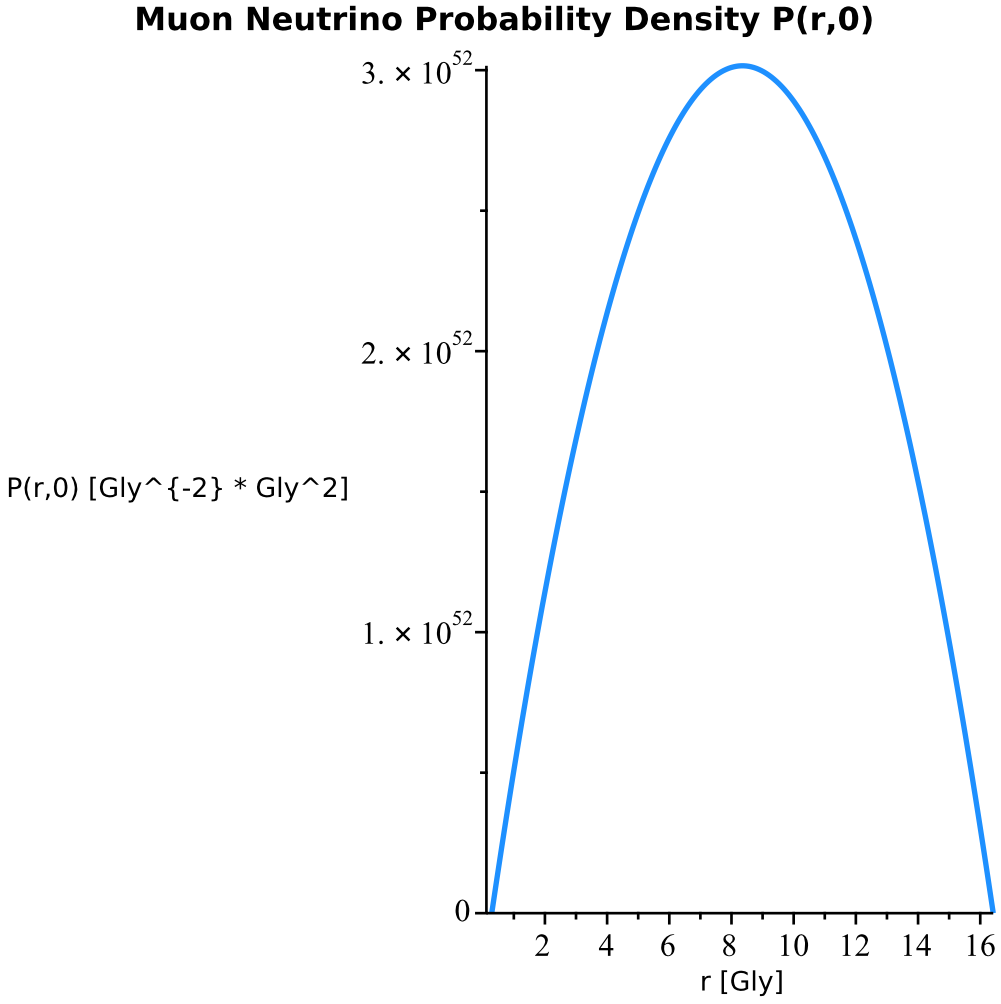
$$alpha_k := 1.910502665 \times 10^{-17}$$

```
g_3(r) = r^{-1/2} * Ai(1.9105e-17 * r)
Plot 4: Airy eigenfunctions [GENERATED]
```

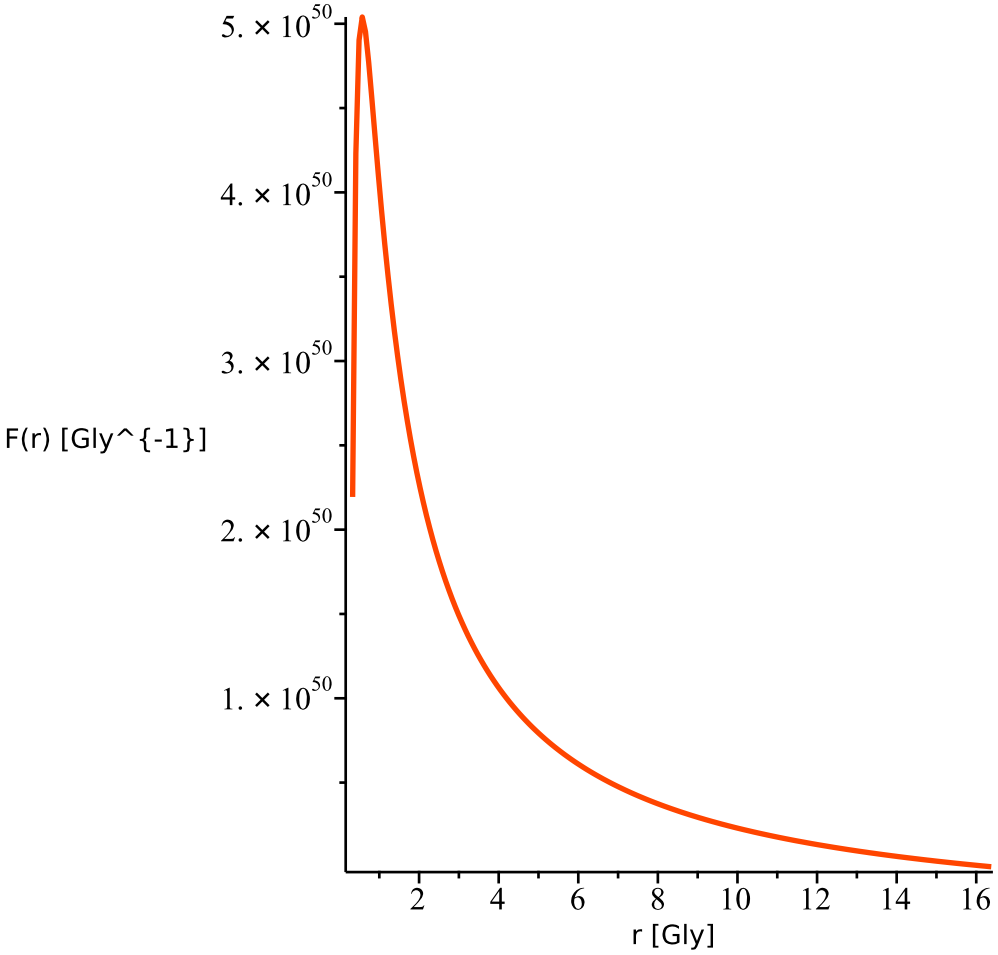
$$tau_vals := [0, 0.5, 1.0, 1.5, 2.0]$$

```
Plot 5: P(r,t) time evolution [GENERATED]

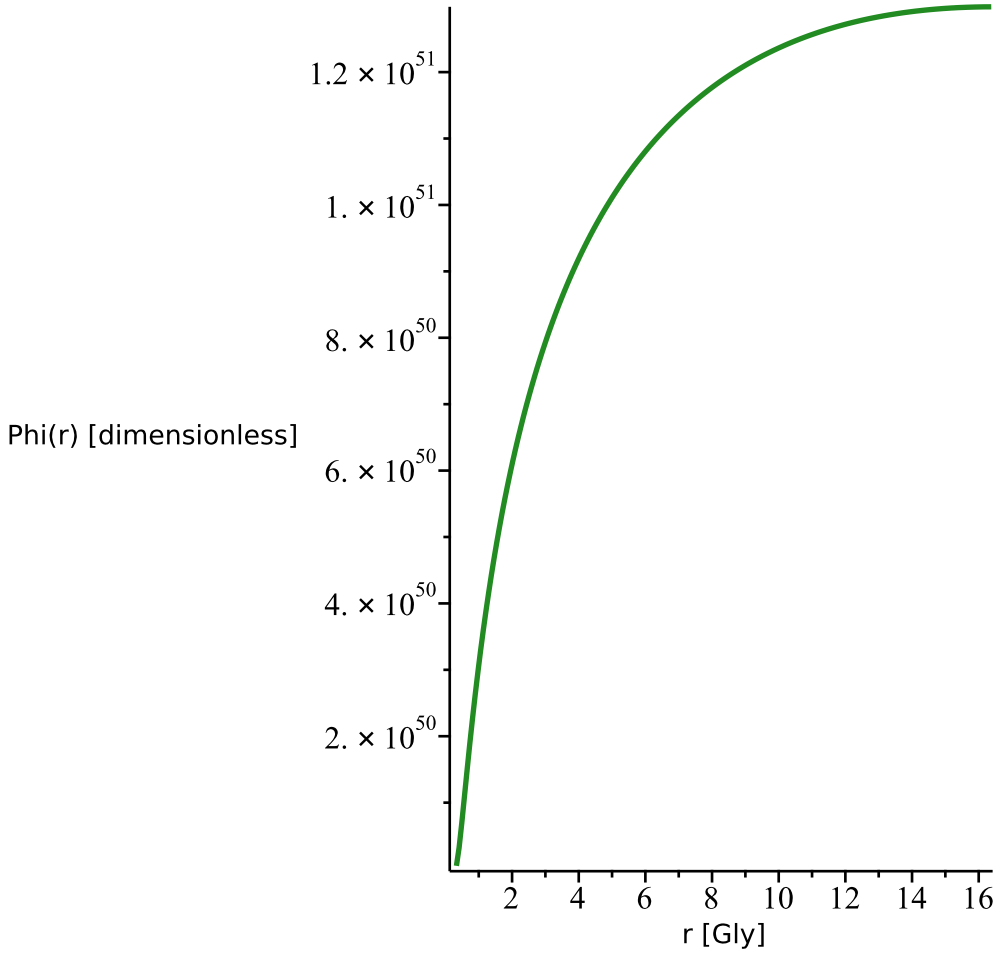
Displaying all 5 plots...
```

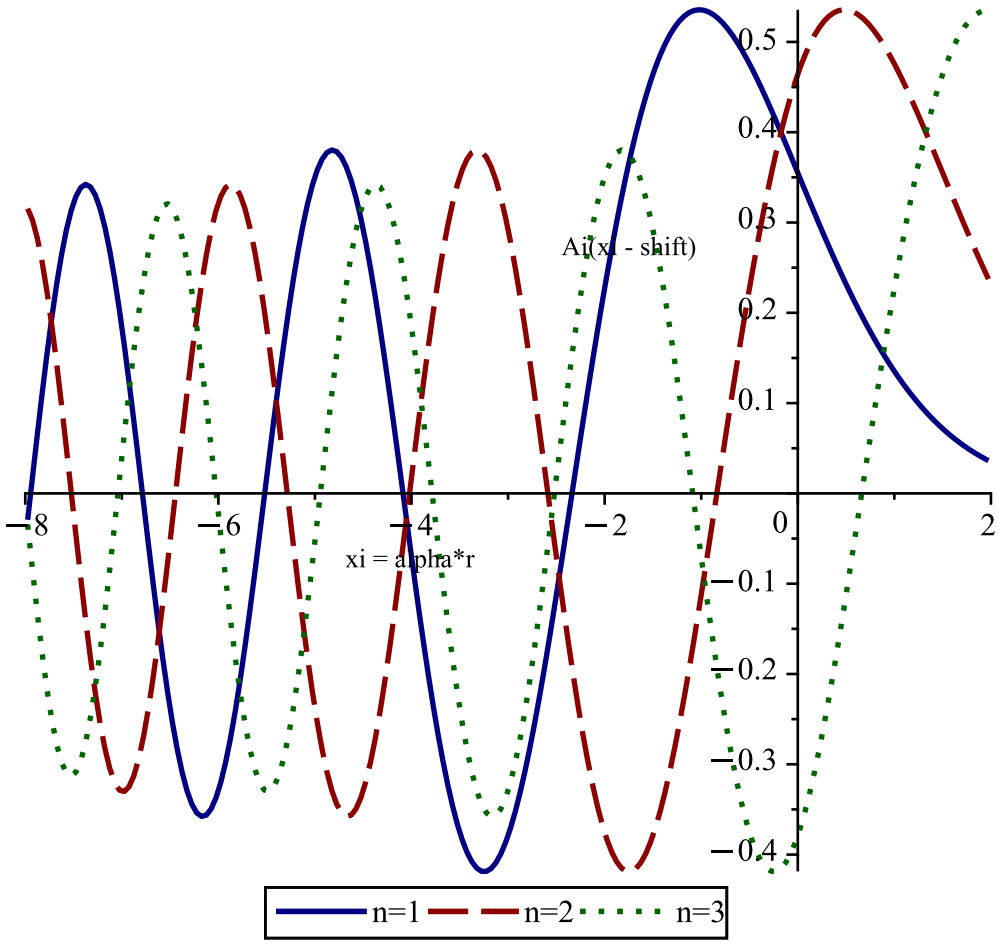


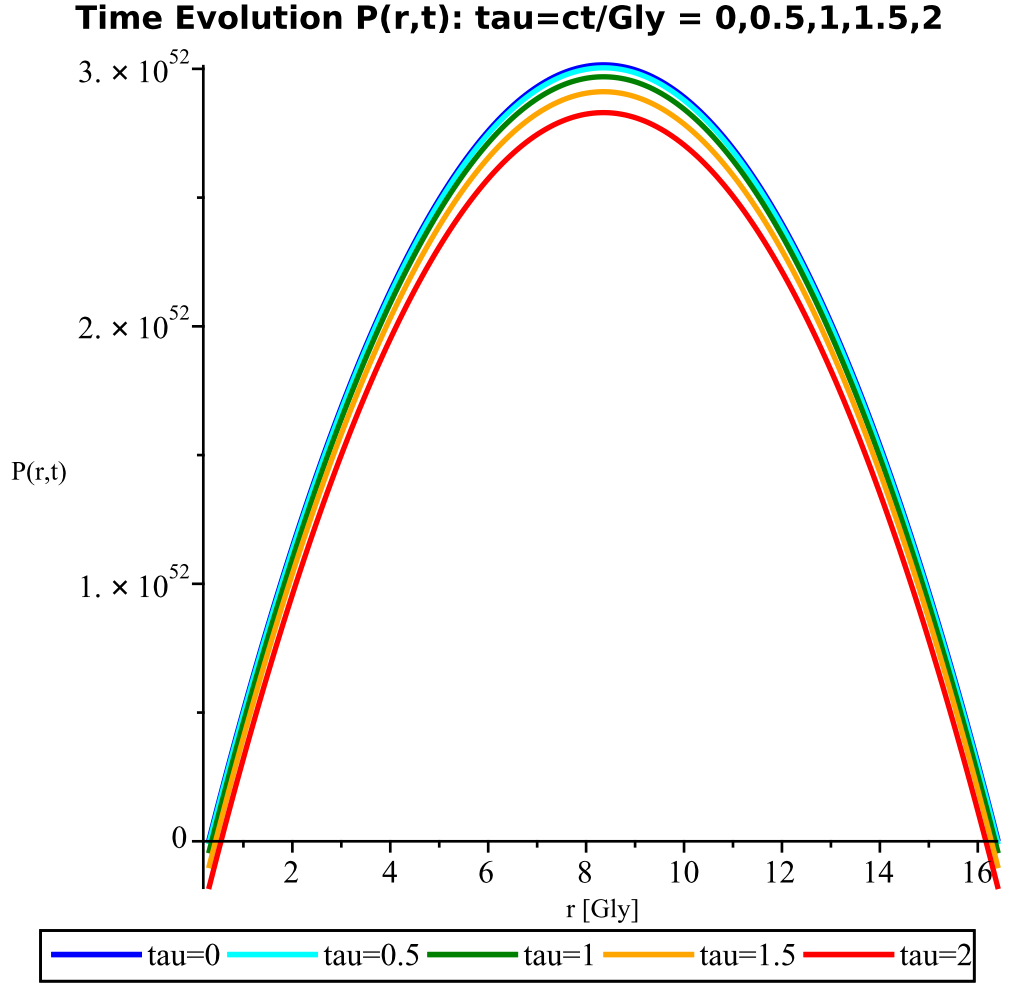
Differential Muon Neutrino Flux F(r) = P(r,0)/(4*pi*r^2)



Cumulative Flux Phi(r) = int_{r_min}^{r} F(r') dr'








```
=== SECTION 10: Verification Summary ===
```

$$P_at_2Rs := 0.$$

```
P(2Rs, 0) = C*(2Rs-r_min)*(2Rs-2Rs) = 0? -> 0.00e+00 [should be 0]
```

$$I_check := 1.372294306 \times 10^{26}$$

```
int F dr (analytic) = 1.372294e+26 m
c/H0 = 1.372294e+26 m
Relative error = 0.00e+00

H0 recovered from int F dr = c/H0:
H0 = 4*pi*c / (C * G_total) = 67.41 km/s/Mpc
```

$$a1 := -2.3381079$$

$$\lambda 1 := 2.028407572 \times 10^{-13}$$

$$\omega 1 := 0.00006081012918$$

```
Lowest eigenfrequency:
lambda_1 = 2.0284e-13 m^{-1}
omega_1 = 6.0810e-05 rad/s
nu_1 = omega_1/(2*pi) = 9.6782e-06 Hz
T_1 = 2*pi/omega_1 = 1.0332e+05 s = 3.2742e-12 Gyr

=== END OF TRANSCRIPT ===
TauTimp / Onduscular Theory — Author: Viorel Popescu
```